

Качество ML-кода и документация

Как менять код так, чтобы он оставался
понятным и работающим?

Стандартизация кода и версионирование данных

Пройдём путь от запуска кода до проверки изменений

Пример занятия: функция считает ошибку предсказаний модели.

- 1. Окружение** Как запустить код с нужными библиотеками?
- 2. Проверки** Как обнаружить ошибку после изменения кода?
- 3. Документация** Как объяснить другому человеку вызов функции?
- 4. Автоматизация** Как не забывать запускать проверки?

Начнём с запуска: код есть, а нужной библиотеки может не быть.

Код не запустится без нужных библиотек

Окружение (env) — Python и установленные для проекта пакеты.

Зависимости — библиотеки, нужные нашему коду.

```
import pandas as pd
```

Этот Python не находит pandas.

```
ModuleNotFoundError:  
No module named 'pandas'
```

У разных проектов могут быть разные наборы и версии пакетов.

Чтобы повторить запуск, запишем требования к зависимостям.

Требования, lock-файл и env — разные вещи

Lock-файл фиксирует выбранные версии пакетов и их зависимостей.

pyproject.toml	Разрешённые версии	Например: pandas >= 2, < 4
uv.lock	Выбранные версии	Точный набор пакетов
.venv/	Локальная установка	Python и библиотеки

Git хранит историю файлов. Требования и lock сохраняем; .venv пересоздаём.

Теперь нужен инструмент, который свяжет эти три части.

uv фиксирует версии, устанавливает их и запускает код

uv — инструмент управления зависимостями и окружением Python-проекта.

<code>uv lock</code>	Записать версии в <code>uv.lock</code>
----------------------	--

<code>uv sync --locked</code>	Установить env по <code>uv.lock</code>
-------------------------------	--

<code>uv run --locked python evaluate.py</code>	Выполнить файл в env
---	----------------------

--locked: если lock отсутствует или устарел — ошибка, а не его обновление.

Окружение готово. Теперь проверим сам код.

Форматтер меняет оформление, сохраняя вычисление

Форматтер автоматически приводит запись кода к единому стилю.

Ruff — инструмент для форматирования и проверки Python-кода.

До форматирования

```
squared=errors**2  
result=squared.mean()
```

```
ruff format evaluate.py
```

errors — разницы между истинными значениями и предсказаниями.

После

```
squared = errors**2  
result = squared.mean()
```

Пробелы исправлены. А сможет ли код вообще выполниться?

Линтер заметит опечатку до запуска программы

Линтер анализирует код без его выполнения и сообщает о подозрительных местах.

```
squared = errors**2  
result = squarred.mean()  
  
ruff check evaluate.py
```

F821 Undefined name `squarred`

Опечатка: squarred → squared

Но даже код без замечаний линтера может считать не то, что нужно.

Тест обнаружит неверную формулу метрики

Тест запускает код и сравнивает результат с ожидаемым.

RMSE — корень из среднего квадрата ошибки предсказаний.

Истина	Прогноз	Ожидаем: $\sqrt{((0^2 + 4^2) / 2)}$
0	0	$= \sqrt{8} \approx 2,83$
0	4	<pre>result = (errors**2).mean() assert np.isclose(result, np.sqrt(8))</pre>

AssertionError — проверка не прошла: $8 \neq \sqrt{8}$.

Мы проверили один расчёт. Какие ещё входы функция должна принимать?

Контракт задаёт, что именно проверять тестами

Контракт функции — требования к входам, результату и поведению при ошибке.

Вход	Пары «истина — прогноз»: числа, без пропусков и бесконечностей; хотя бы одна пара.
Результат	RMSE — одно число. Для нашего примера: $\sqrt{8}$.
Ошибка входа	Понятная ошибка вместо некорректной метрики.

Если передаём пары в таблице, нужно проверить и её содержимое.

Две таблицы одного типа могут содержать разные данные

DataFrame — таблица pandas со строками и именованными колонками.

Допустимый вход

y_true	y_pred
0	0
0	4

Нарушение: NaN — пропуск

y_true	y_pred
0	0
0	NaN

Обе — DataFrame. Справа пропуск: наш контракт его не допускает.

Требования определены. Где прочитает их пользователь функции?

Docstring объясняет, как вызвать функцию

Docstring — строка документации сразу после объявления функции.

```
def rmse(frame):  
    """Вычислить RMSE по таблице.  
  
    Args:  
        frame: Колонки y_true и y_pred; непустые,  
              конечные числовые значения.  
  
    Returns:  
        Ошибка в единицах целевой величины (float).  
    """
```

Фрагмент без тела функции. Args — вход, Returns — результат.

Из этого описания можно собрать страницу документации.

Sphinx собирает страницу с описанием функции

Sphinx — программа для сборки документации из текстов и описаний в коде.

Результат сборки Sphinx

API — интерфейс
использования кода:
какие функции можно
вызвать,
что им передать и получить.

`rmse`(*frame*)

Вычислить RMSE по таблице.

Параметры:

`frame` – Колонки `y_true` и `y_pred`; непустые, конечные числовые значения.

Результат:

Ошибка в единицах целевой величины (float).

Код, тесты и документация готовы. Как запускать проверки автоматически?

Hook запускает проверку перед созданием коммита

Git hook — программа, которую Git запускает при заданном событии.

Коммит сохраняет подготовленные изменения в истории проекта.

Событие	Попытка коммита
Действие	Проверка Ruff
Результат	Ошибка → коммит остановлен

`git add` подготавливает исправления к коммиту; затем коммит повторяем.

CI — непрерывная интеграция: изменения регулярно объединяют и автоматически проверяют. Общие проверки обычно запускают на сервере.

Автоматизируем уже знакомые проверки, а не заменяем их новым инструментом.

Выбираем проверку по проблеме, которую хотим найти

Что вы проверите первым в каждой ситуации?

Не импортируется pandas

Python, env и установленные зависимости

Опечатка в имени переменной

Линтер

RMSE = 8 вместо $\sqrt{8}$

Тест с известным правильным ответом

Непонятно, что передать функции

Контракт и документация

Забываем запускать проверки

Автоматический запуск через hooks / CI

Далее по теме — хранение и версионирование данных.

pip, conda и uv решают задачи установки по-разному

Менеджер пакетов устанавливает библиотеки и учитывает их зависимости.

pip	Установка Python-пакетов в выбранный Python	<code>python -m pip install pandas</code>
conda	Окружения, Python и пакеты из каналов conda	<code>conda install pandas</code>
uv	Python-проект, lock-файл и его окружение	<code>uv add pandas</code>

Это альтернативные сценарии. На занятии подробно разбираем один — uv.

Google-style и NumPy-style меняют запись, а не смысл

Два стиля описания параметров и результата: отличаются отступы и разделители.

Google-style

Args:

 frame: Таблица предсказаний.

Returns:

 float: RMSE.

NumPy-style

Parameters

frame

 Таблица предсказаний.

Returns

float

 RMSE.

В одном проекте выбираем один стиль. Sphinx умеет разбирать оба.

Общие правила Ruff записываем в `pyproject.toml`

PEP 8 — рекомендации по стилю Python-кода: имена, отступы, оформление.

```
[tool.ruff]
target-version = "py313"
line-length = 88
```

```
[tool.ruff.lint]
select = [
    "E4", "E7", "E9", "F", "I", "B",
]
```

F — имена и импорты

I — порядок импортов

B — подозрительные конструкции

py313 — проверка кода
для Python 3.13.

Правила согласует команда. Конфигурация делает их одинаковыми для всех.